

Prefix Function y Aho-Corasick

Ignacio Muñoz

`ignacio.munoz@progcomp.cl`

Prefix Function (KMP)

1 · Prefix Function

2 · Aho-Corasick



Problema: contar ocurrencias de un patrón

Dado un texto S y un patrón T , contar cuántas veces T aparece como substring de S .

Ejemplo: $S = \text{blacarcarbla}$, $T = \text{car} \rightarrow 2$ ocurrencias.

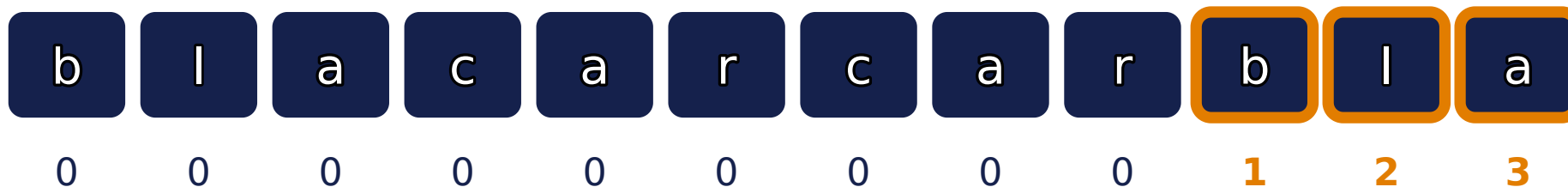
A la mala: comparar T contra cada posición de S cuesta $O(|S| \cdot |T|)$; con $|S|, |T| \leq 10^5$, inviable.

Idea de KMP: cuando una comparación falla, no reiniciar desde cero: usar lo que **ya sabemos** que coincide.



Prefix function: definición

Para un string s , $\pi(i)$ es el largo del mayor **prefijo propio** de $s[0..i]$ que también es **sufijo** de $s[0..i]$.



$\pi(i)$ para cada posición i



"bla" (prefijo) = "bla" (sufijo) → borde de largo 3

$s = \text{blacarcarbla}$: el prefijo **bla** reaparece como sufijo, y π lo detecta al llegar a esas posiciones.

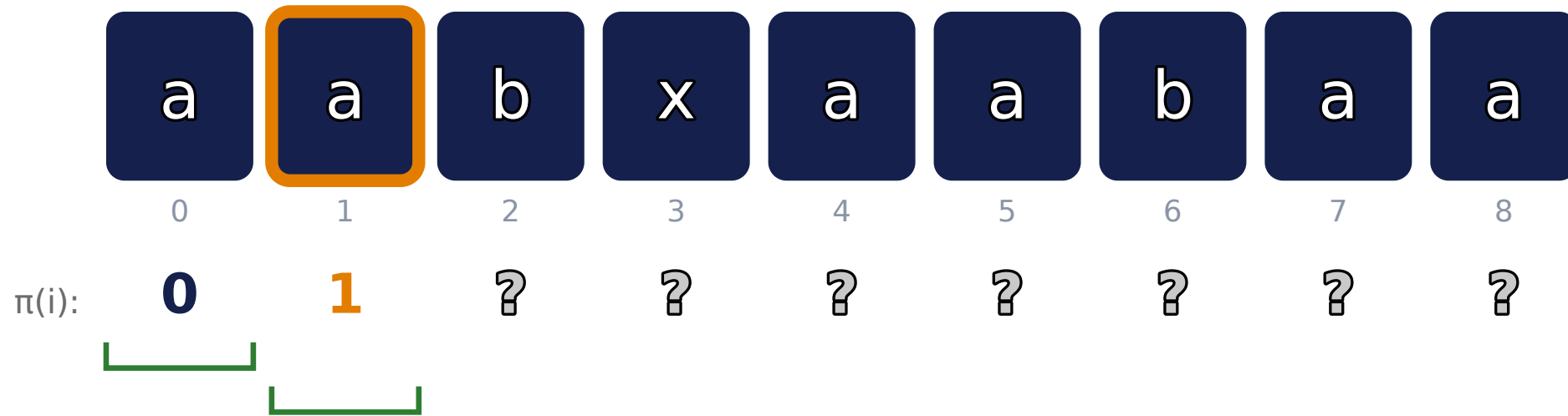


Ejemplo: aabxaabaa

	a	a	b	x	a	a	b	a	a
	0	1	2	3	4	5	6	7	8
$\pi(i)$:	0	?	?	?	?	?	?	?	?



Ejemplo: aabxaabaa





Ejemplo: aabxaabaa

	a	a	b	x	a	a	b	a	a
	0	1	2	3	4	5	6	7	8
$\pi(i)$:	0	1	0	?	?	?	?	?	?

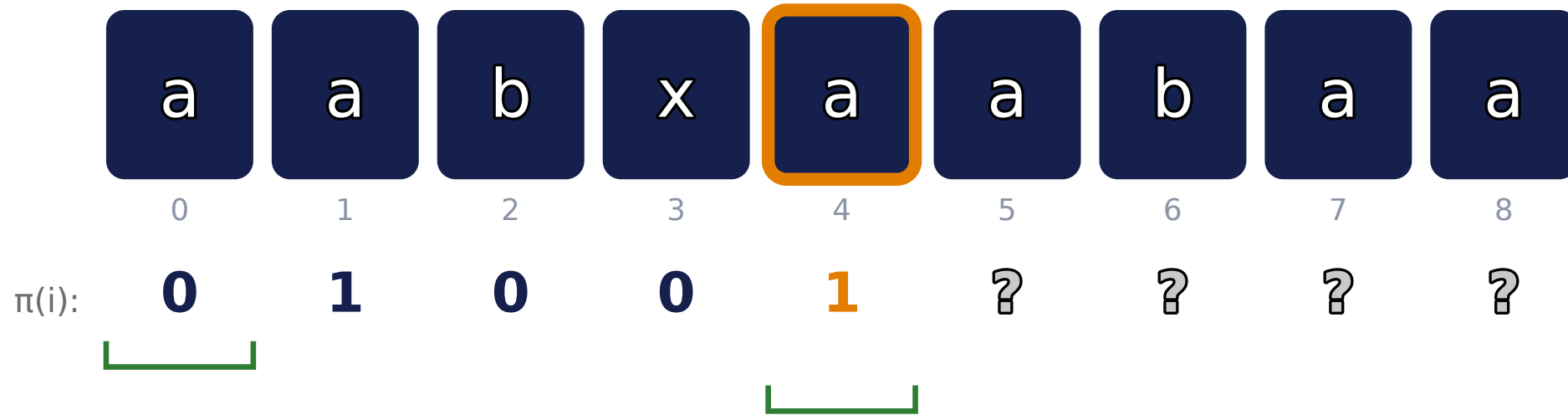


Ejemplo: aabxaabaa

	a	a	b	x	a	a	b	a	a
	0	1	2	3	4	5	6	7	8
$\pi(i)$:	0	1	0	0	?	?	?	?	?

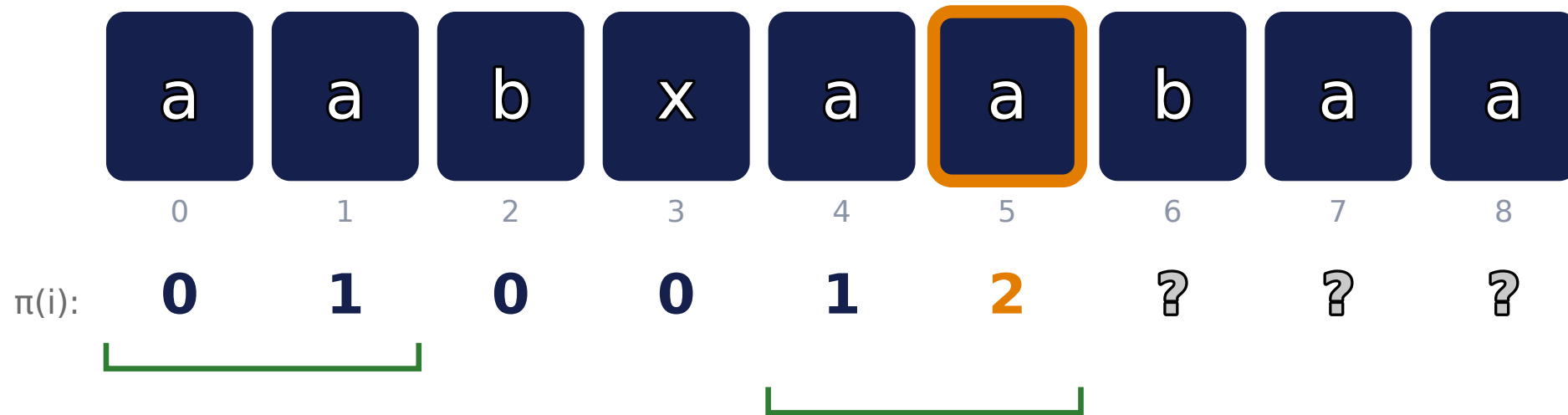


Ejemplo: aabxaabaa



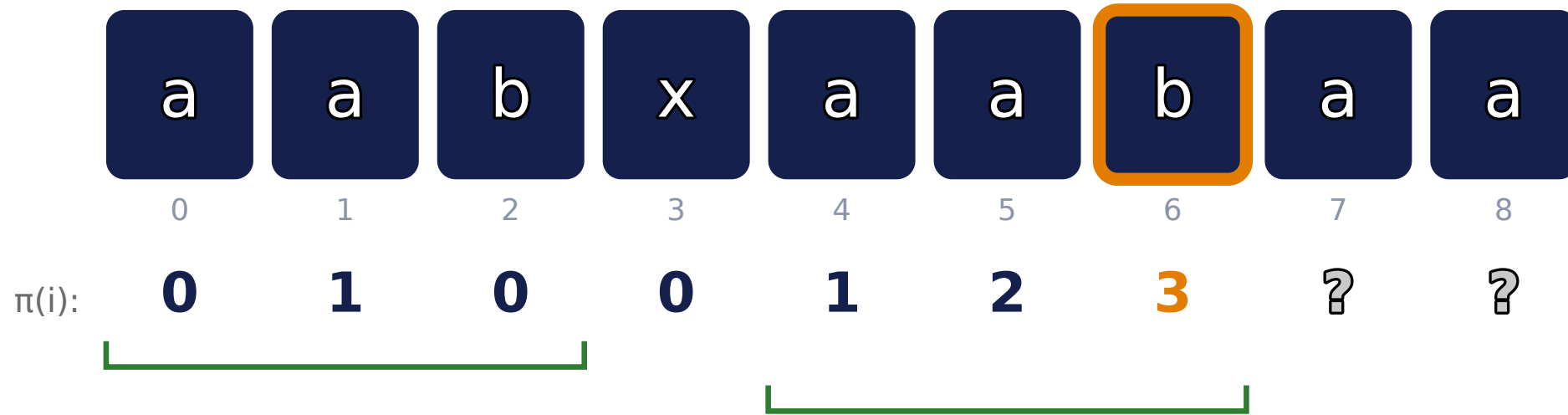


Ejemplo: aabxaabaa



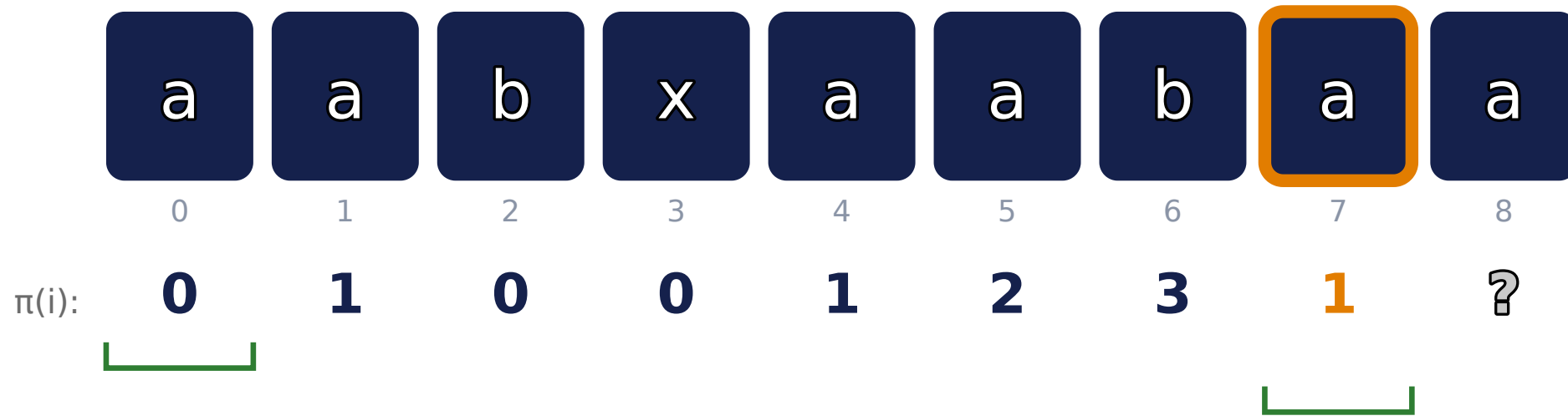


Ejemplo: aabxaabaa



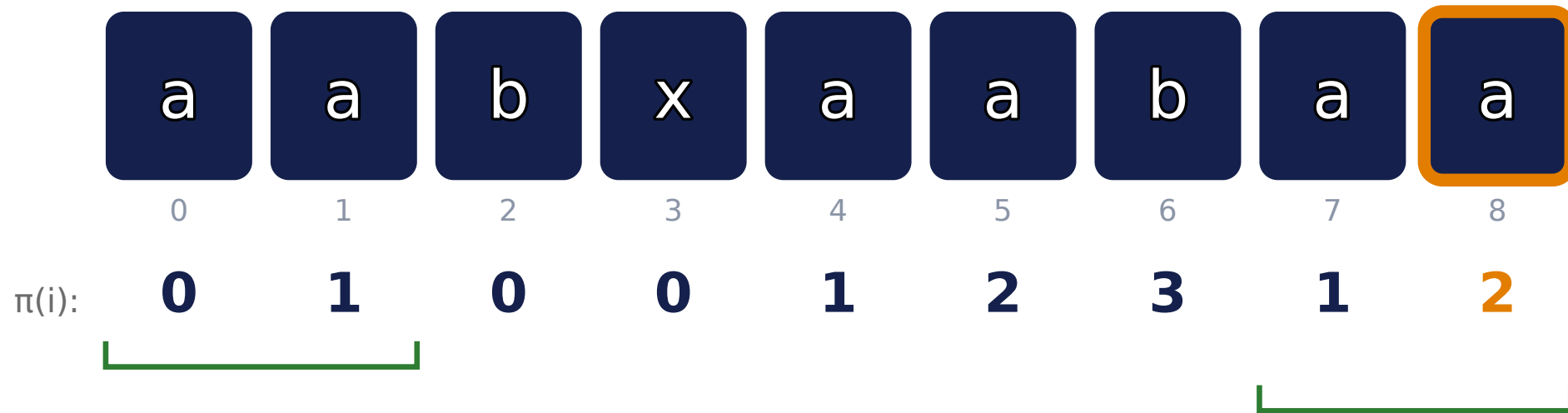


Ejemplo: aabxaabaa



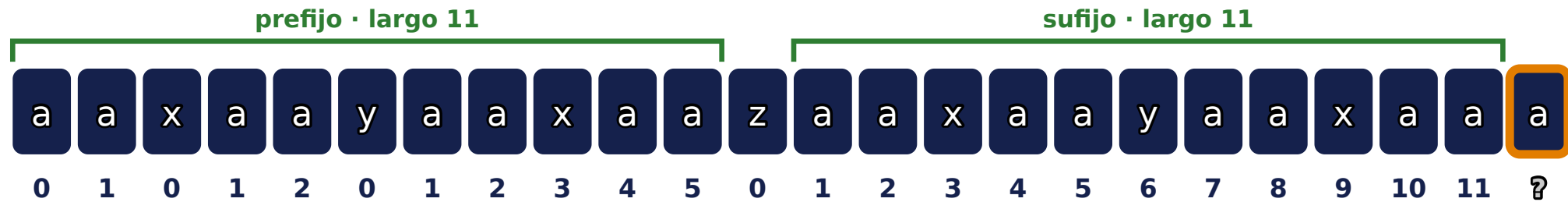


Ejemplo: aabxaabaa



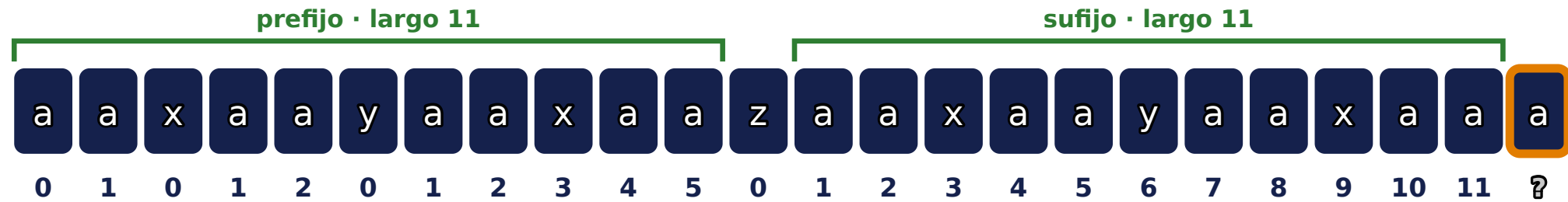
Ejemplo: agregando la última letra

Todo lo demás ya está calculado ($\pi(0)$ a $\pi(22)$). Falta $\pi(23)$.



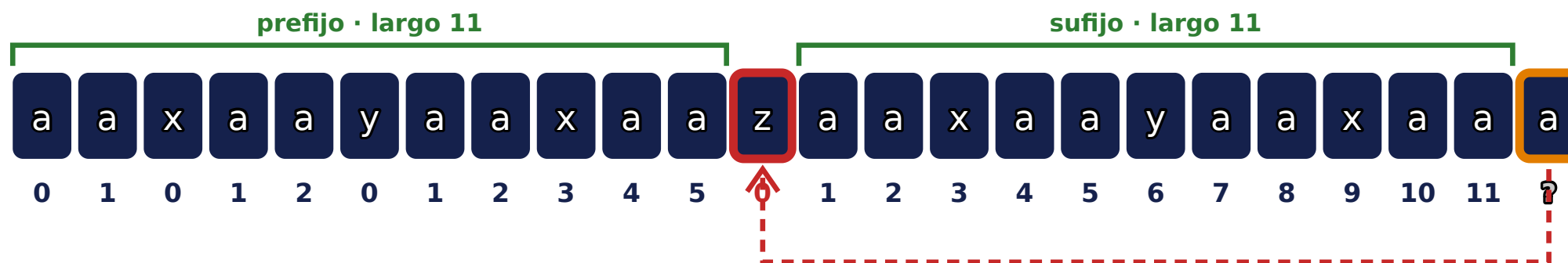


Candidato: borde de largo 11





¿Coincide? $j = 11$





Candidato: borde de largo 5





¿Coincide? $j = \pi(10) = 5$





Candidato: borde de largo 2

prefijo · largo 2

sufijo · largo 2





¿Coincide? $j = \pi(4) = 2$





Candidato: borde de largo 1

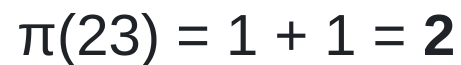
prefijo · largo 1



a	a	x	a	a	y	a	a	x	a	a	z	a	a	x	a	a	y	a	a	x	a	a	a
0	1	0	1	2	0	1	2	3	4	5	0	1	2	3	4	5	6	7	8	9	10	11	?

sufijo · largo 1







Cómo se calcula: un puntero que no retrocede

Se mantiene un puntero j (candidato a coincidencia). Al comparar $s[i]$ con $s[j]$:

- Si coinciden, j avanza: la coincidencia crece.
- Si no, j **salta hacia atrás** usando $\pi(j - 1)$ (no vuelve a comparar desde cero, reusa un borde ya conocido).



Análisis amortizado

j crece a lo sumo n veces en total (una por iteración de i), así que también puede *decrecer* a lo sumo n veces: el ciclo interno corre $O(n)$ en total, no por iteración.



Código: Prefix Function

```
vector<int> prefix_function(const string &s) {  
    int n = s.size();  
    vector<int> pi(n, 0);  
    for (int i = 1; i < n; i++) {  
        int j = pi[i - 1];  
        while (j > 0 && s[i] != s[j])  
            j = pi[j - 1];  
        if (s[i] == s[j])  
            j++;  
        pi[i] = j;  
    }  
    return pi;  
}
```

$O(n)$ amortizado: el `while` interno nunca deshace más de lo que `j` alcanzó a avanzar.



KMP search: buscar el patrón

Truco: concatenar $T \# S$ (un separador que no aparece en ninguno) y correr `prefix_function` sobre el resultado. Donde $\pi(i) = |T|$, hay un match.

c	a	r	#	b	l	a	c	a	r	c	a	r	b	l	a
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

$T \# S = \text{"car\#blacarcarbla"}$ (16 caracteres)

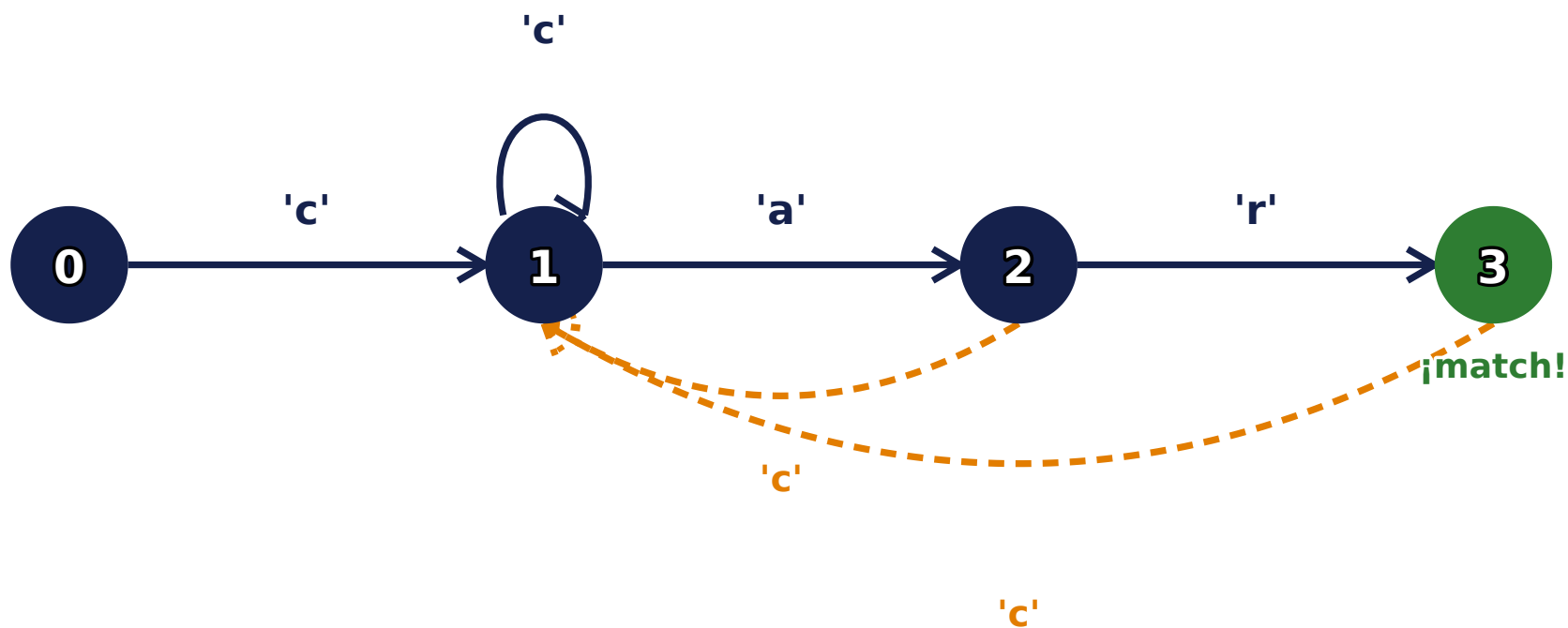
$\pi(i)$	0	0	0	0	0	0	1	2	3	1	2	3	0	0	0
----------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

$\pi(i) = |T| = 3 \rightarrow \text{match}$. Posición en $S = i - 2 \cdot |T| \rightarrow 3$ y 6



Autómata

El mismo ejemplo ($T = \text{car}$) se puede ver como un **autómata**: un estado por cada prefijo de T ya matcheado, y una transición por cada carácter leído.



Si el carácter que sigue **no calza**, no siempre se vuelve al estado 0: una `'c'` salta directo al estado 1, porque ya hay un prefijo de largo 1 matcheado. Cualquier otro



Bordes de un string

Todo **borde** de s (prefijo que también es sufijo) se obtiene siguiendo la cadena $\pi(n-1) \rightarrow \pi(\pi(n-1)-1) \rightarrow \dots \rightarrow 0$.

Ejemplo: $s = \text{abccabccabcc}$, $\pi = [0, 0, 0, 1, 2, 3, 4, 5, 6]$.

$$\pi(8) = 6 \rightarrow \pi(5) = 3 \rightarrow \pi(2) = 0$$

Los bordes no triviales son 6 ("abccabcc") y 3 ("abc").

Para practicar: CSES 1732 (Finding Borders).



Período de un string

Si $n - \pi(n-1)$ **divide** a n , ese valor es el **período** más chico del string.

Ejemplo: $s = \text{abcabcabc}$, $n = 9$, $\pi(n-1) = 6$.

$$n - \pi(n - 1) = 9 - 6 = 3, \quad 3 \mid 9 \Rightarrow \text{período } 3 \text{ ("abc").}$$

Para practicar: CSES 1733 (Finding Periods).



E. Compress Words

Aho-Corasick

1 · Prefix Function

2 · **Aho-Corasick**



Problema: k patrones a la vez

Ahora no hay un solo patrón: hay k patrones $\{T_1, \dots, T_k\}$ y un texto S . Hay que encontrar **todas** las ocurrencias de **cada** patrón en S .

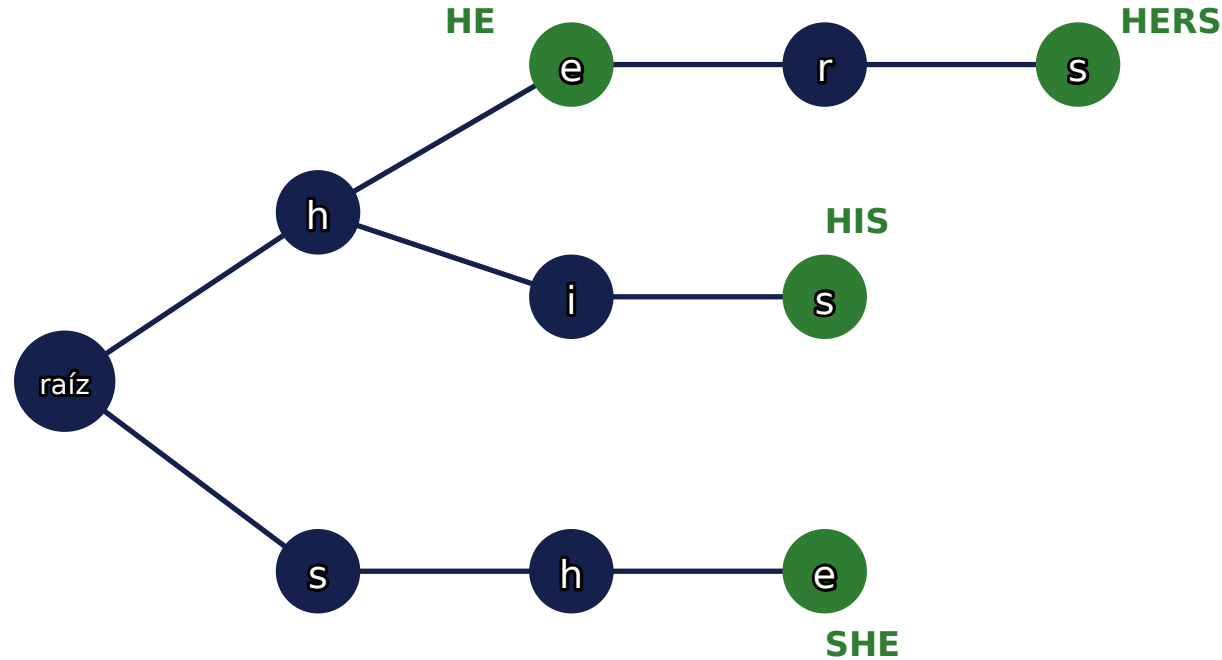
Ejemplo: patrones $\{\text{he}, \text{she}, \text{his}, \text{hers}\}$, buscarlos en un texto largo.

A la mala: correr KMP k veces cuesta $O(k \cdot (|S| + |T_i|))$; con k y $|S|$ grandes, no alcanza.

¿Se puede generalizar la prefix function a varios patrones al mismo tiempo?



Idea: unir los patrones en un trie

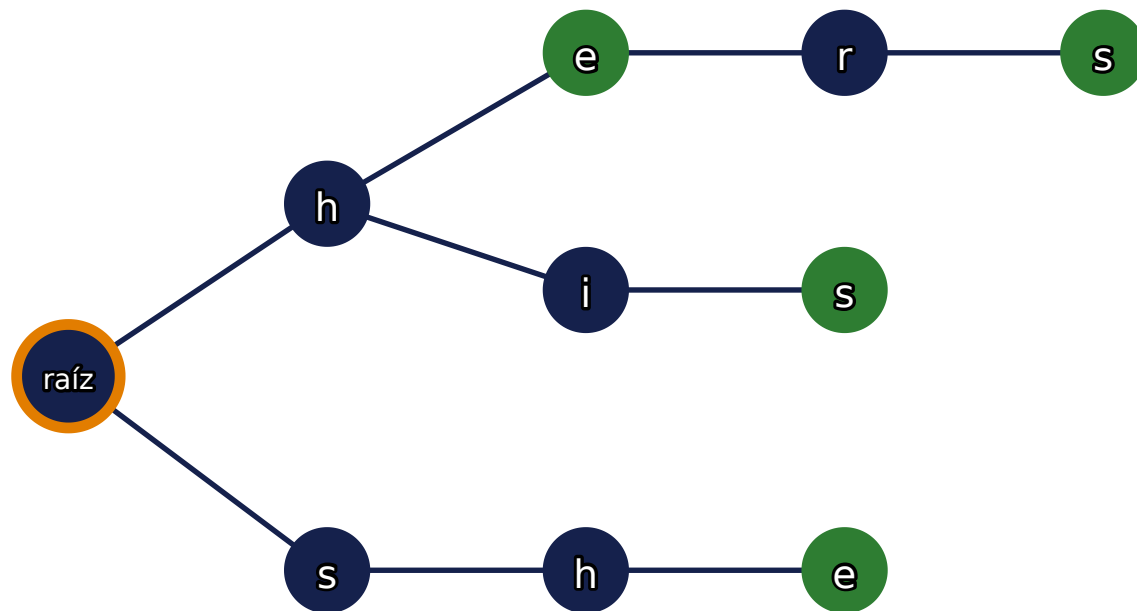


nodo verde

= termina un patrón ahí (end)

Cada patrón es un **camino desde la raíz**; los prefijos comunes (como **h**) se comparten. Construcción: $O(\sum |T_i|)$.

Recorriendo el trie, letra por letra

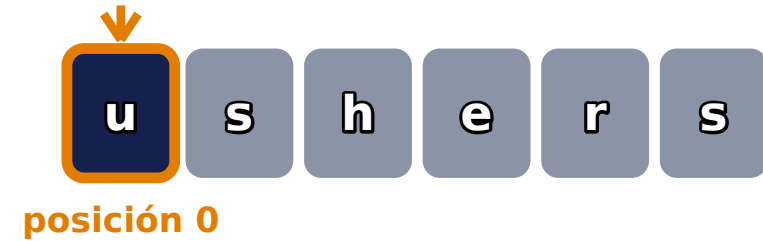
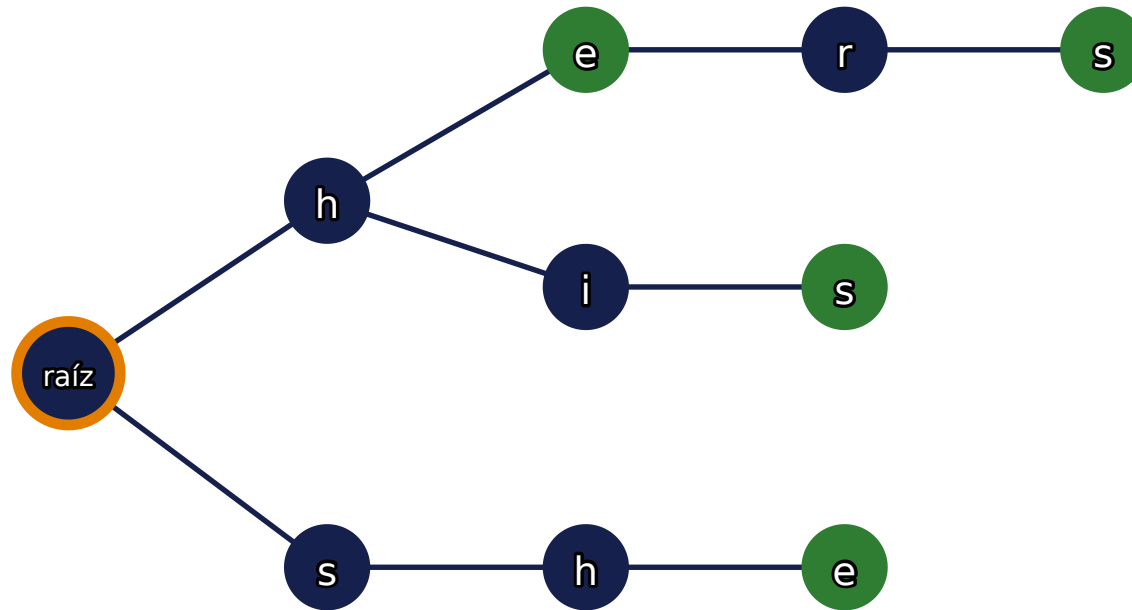


u s h e r s

un solo puntero recorre $S = \text{"ushers"}$ sobre el autómata



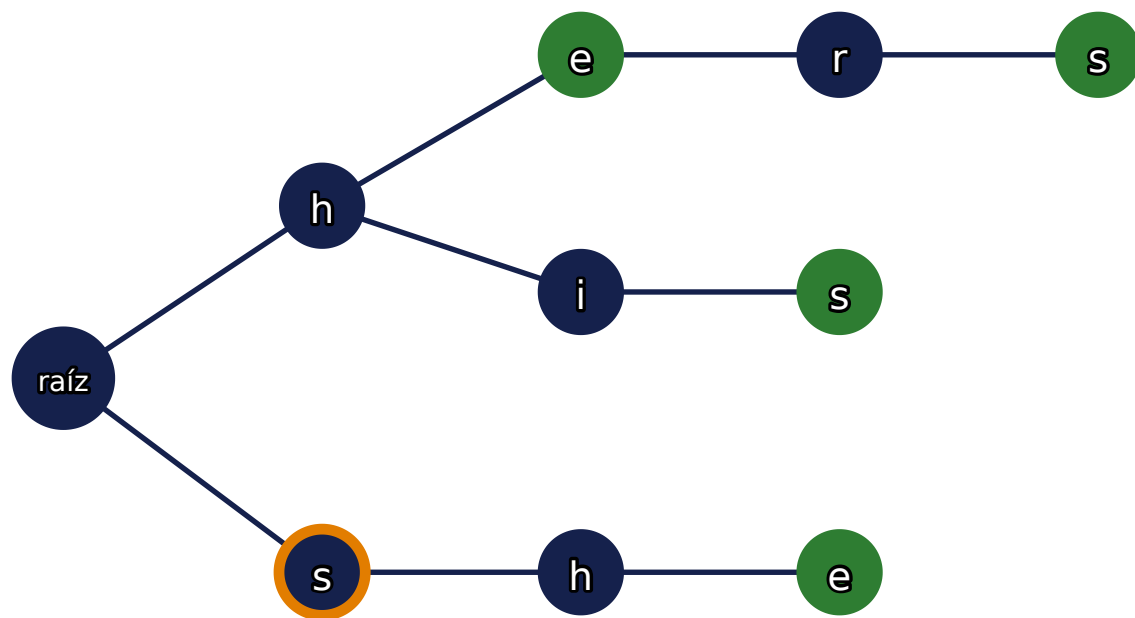
Paso: leer 'u' (posición 0)



la raíz no tiene hijo 'u' → se queda en la raíz, sin match



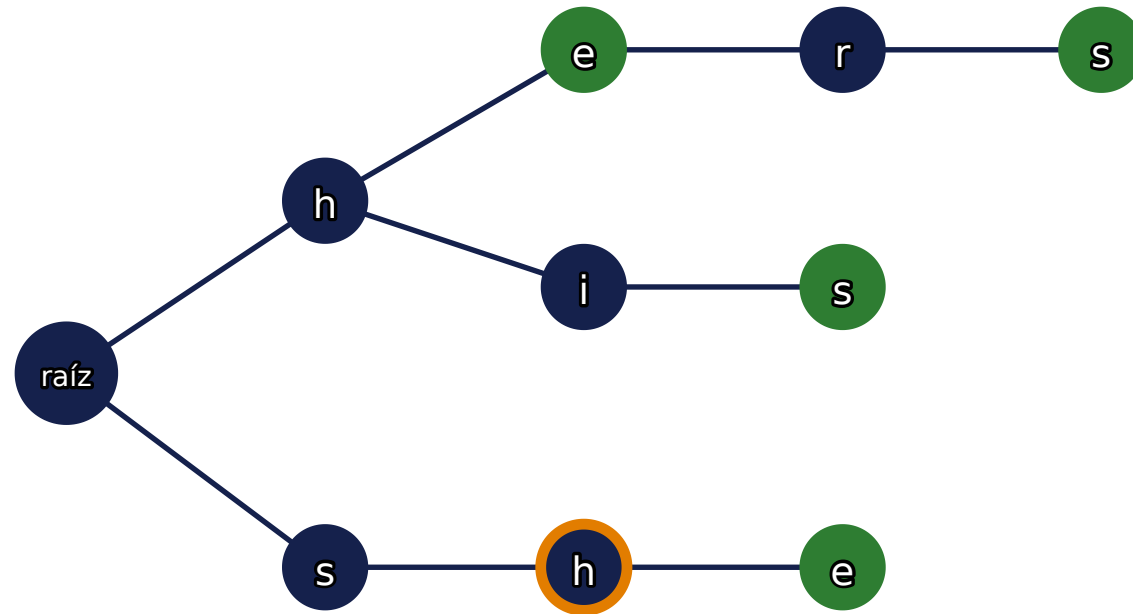
Paso: leer 's' (posición 1)



raíz tiene hijo 's' → avanza al nodo "s"



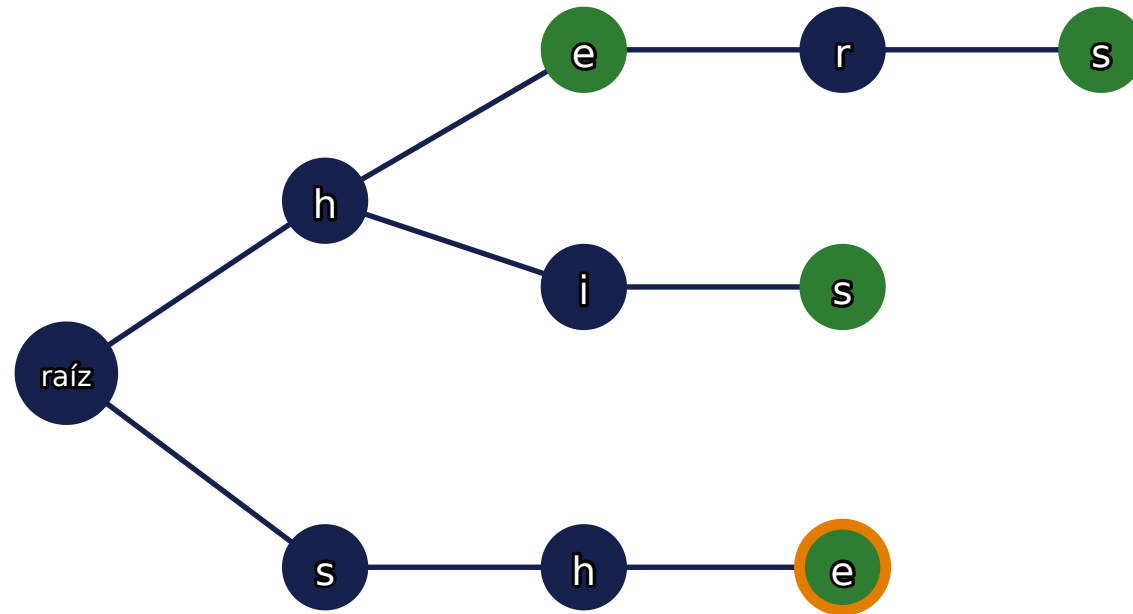
Paso: leer 'h' (posición 2)



"s" tiene hijo 'h' → avanza al nodo "sh"

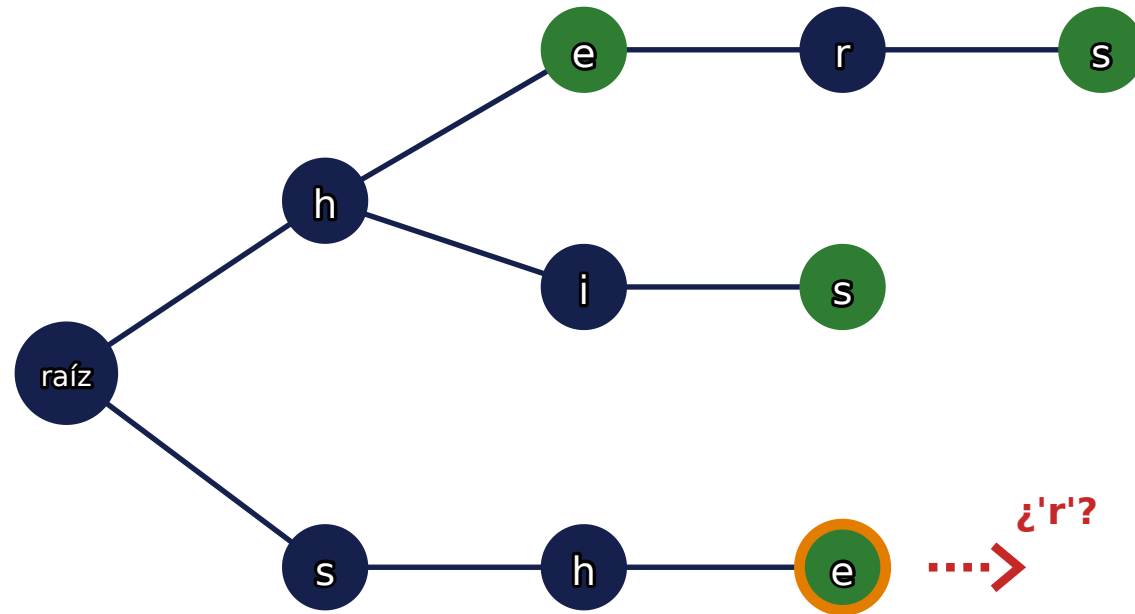


Paso: leer 'e' (posición 3) → ¡match!



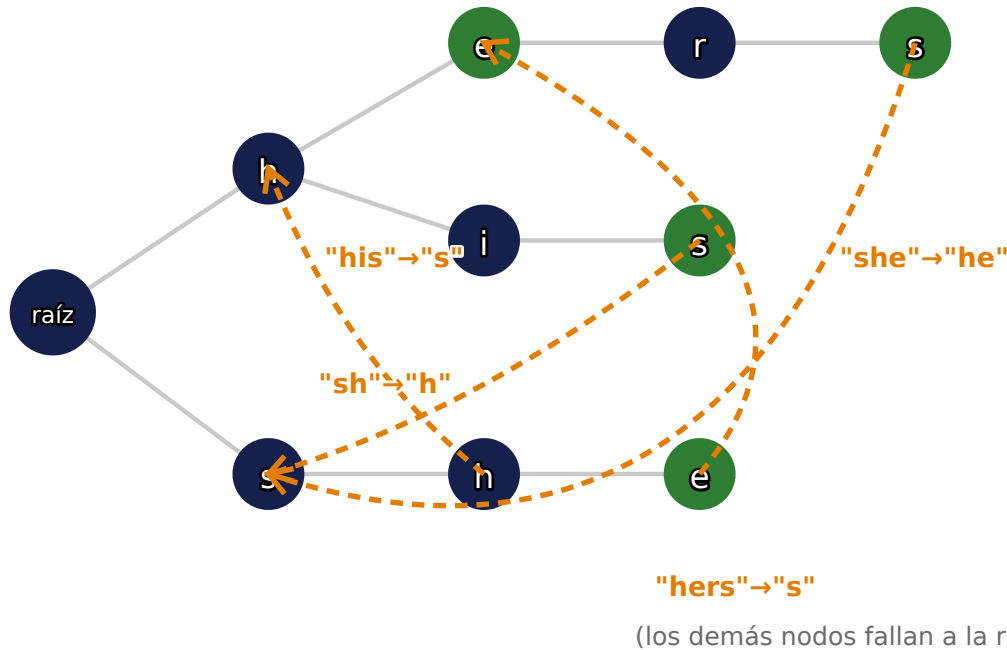


Paso: leer 'r' (posición 4) → sin hijo directo





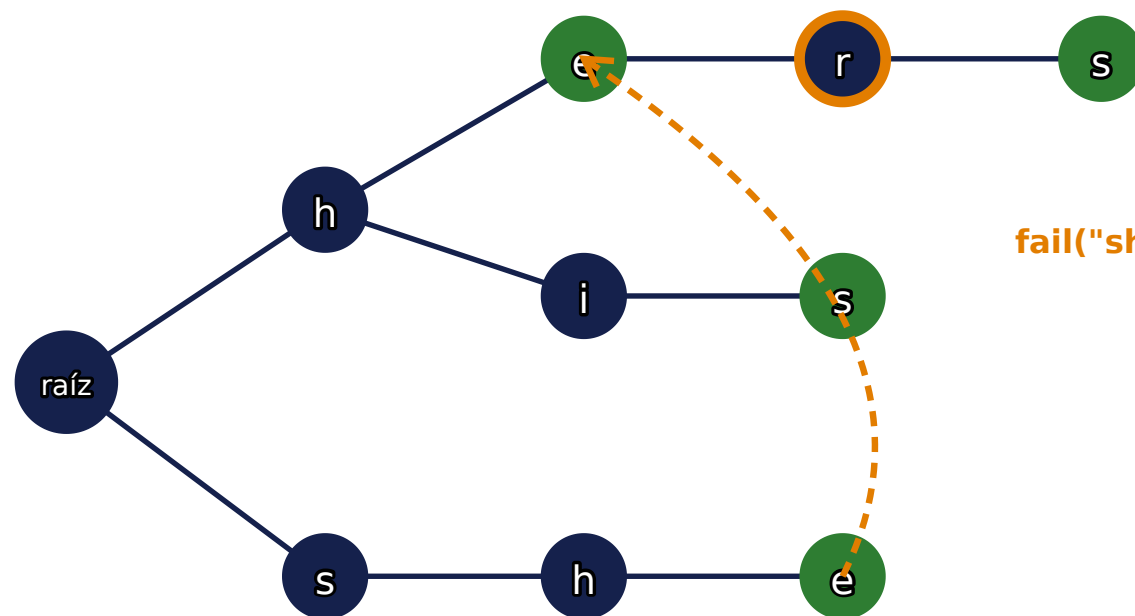
Failure links: la prefix function del trie



$\text{fail}(v)$ = el nodo del trie con el **sufijo propio más largo** de la cadena de v que también es prefijo de algún patrón. Es la misma idea que π , pero ahora sobre un árbol en vez de una sola cadena.



Paso: leer 'r' (posición 4) → seguir el fail link



fail("she") = "he"

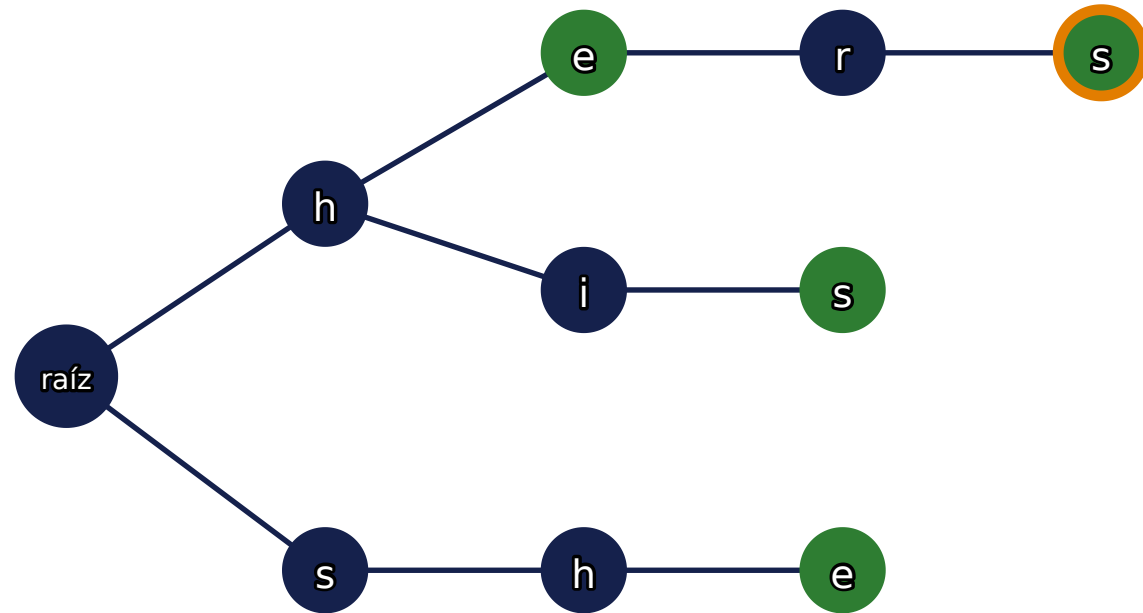


posición 4

"he" (end) → reporta: HE
"he" sí tiene hijo 'r' → avanza a "her"
(no es end, aún no hay match aquí)

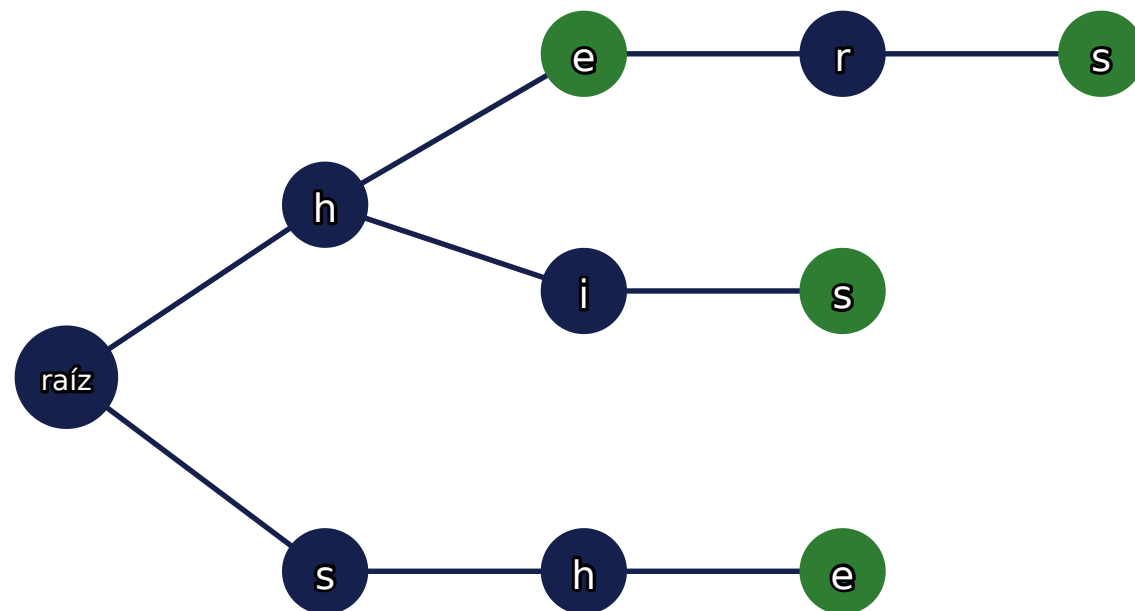


Paso: leer 's' (posición 5) → ¡match!





Un solo recorrido, todos los matches



u s h e r s

S = "ushers"
SHE, HE, HERS

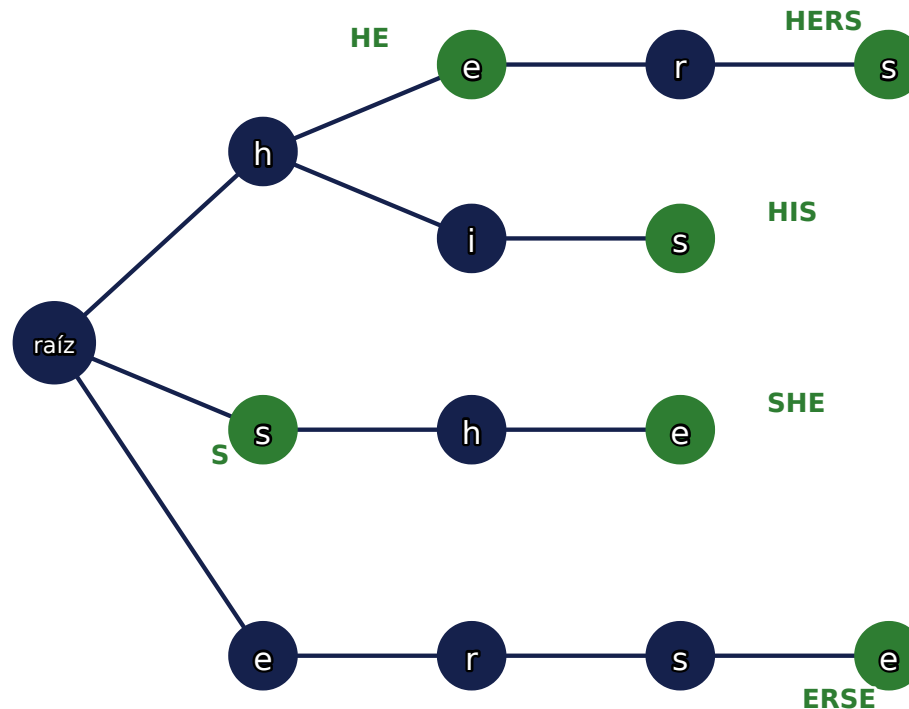
un recorrido lineal, sin retroceder

"she" contiene "he" como sufijo: el fail-link lo reporta gratis. El fail-chain se sigue **cada vez** que se llega a un nodo, no solo cuando falta una transición.

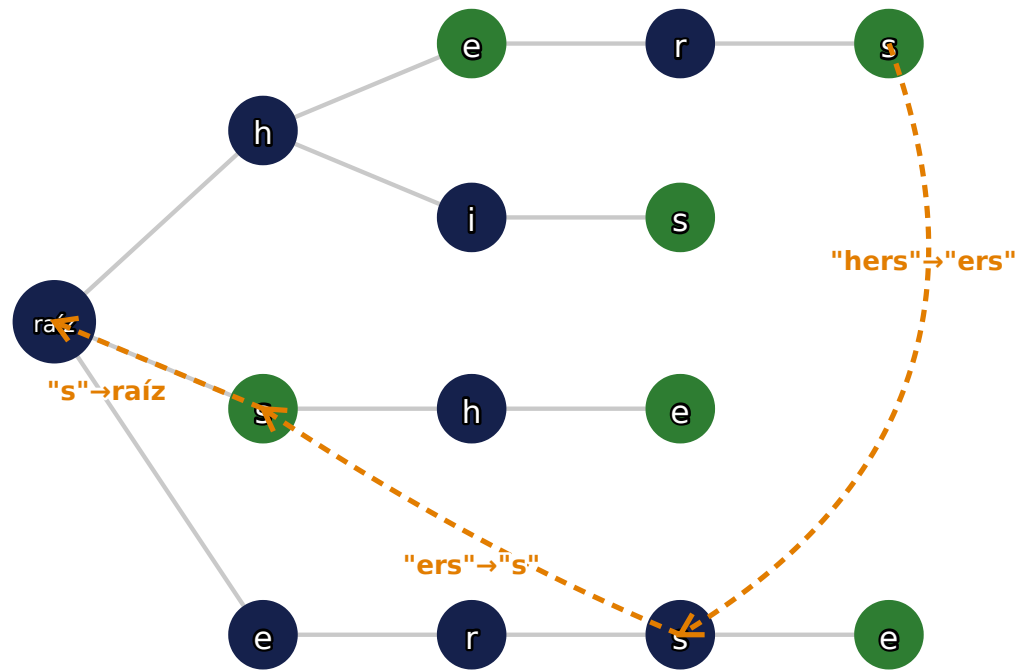


Otro ejemplo: agregando "erse" y "s"

Al conjunto {he, she, his, hers} le agregamos dos patrones nuevos: **erse** y **s** (un solo carácter).



Failure links desde la "s" de "hers"



Antes de agregar **erse**, $\text{fail}(\text{hers})$ apuntaba directo a **s**. Con **erse** en el conjunto, "ers" (sufijo propio de "hers") es ahora prefijo de un patrón, así que el fail link se **alarga**: $\text{hers} \rightarrow \text{ers} \rightarrow \text{s} \rightarrow \text{raíz}$.



Código: el vértice

```
struct Vertex {  
    int next[26], go[26];    // next: aristas del trie; go: autómata (memo)  
    int p, link = -1, exit = -1, cnt = -1;  
    vector<int> leaf;        // ids de patrones que terminan aquí  
    char pch;  
    Vertex(int p = -1, char ch = '$') : p(p), pch(ch) {  
        for (int i = 0; i < 26; i++) next[i] = -1, go[i] = -1;  
    }  
};  
vector<Vertex> t(1);
```

p / pch : el padre y la letra usada para llegar aquí, para calcular link bajo demanda.
link, exit, cnt empiezan en -1 = "no calculado".



Código: insertar un patrón

```
void add(string &s, int id) {  
    int v = 0;  
    for (char ch : s) {  
        int c = ch - 'a';  
        if (t[v].next[c] == -1) {  
            t[v].next[c] = t.size();  
            t.emplace_back(v, ch);  
        }  
        v = t[v].next[c];  
    }  
    t[v].leaf.push_back(id);  
}
```

Igual que un trie normal. `id` identifica **cuál** patrón termina en cada nodo (puede haber más de uno).



Código: fail link, recursivo con memo

```
int get_link(int v) {  
    if (t[v].link == -1) {  
        if (v == 0 || t[v].p == 0) t[v].link = 0;  
        else t[v].link = go(get_link(t[v].p), t[v].pch);  
    }  
    return t[v].link;  
}
```

Un solo padre por nodo: la recursión sube por `p` hasta la raíz, sin ningún `for`.



Código: la función goto, memoizada

```
int go(int v, char ch) {  
    int c = ch - 'a';  
    if (t[v].go[c] == -1) {  
        if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];  
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);  
    }  
    return t[v].go[c];  
}
```

`next[]` es el trie puro; `go[]` cachea la transición completa del autómata la primera vez que se pide. `go` y `get_link` se llaman entre sí, por eso `go` necesita declaración adelantada.



Código: próximo match, recursivo con memo (opcional)

```
int next_match(int v) {  
    if (t[v].exit == -1) {  
        if (t[get_link(v)].leaf.size()) t[v].exit = get_link(v);  
        else t[v].exit = v == 0 ? 0 : next_match(get_link(v));  
    }  
    return t[v].exit;  
}
```

Salta por el fail-chain directo al próximo nodo que **sí** tiene un patrón terminando ahí, saltándose los nodos vacíos.



Código: contar matches en la posición, recursivo con memo (opcional)

```
int cnt_matches(int v) {  
    if (t[v].cnt == -1)  
        t[v].cnt = v == 0 ? 0 : t[v].leaf.size() + cnt_matches(get_link(v));  
    return t[v].cnt;  
}
```

Suma cuántos patrones terminan en **v** y en cada sufijo suyo, subiendo por el fail-chain. Se llama una vez por carácter al recorrer el texto.



Ejemplo de uso

```
for (int i = 0; i < (int)patrones.size(); i++) add(patrones[i], i);

int v = 0;
long long total = 0;
for (char ch : texto) {
    v = go(v, ch);
    total += cnt_matches(v);           // solo el conteo total de matches
}
```

Si además necesitas **cuáles** patrones matchean en cada posición (no solo cuántos), se usa `next_match` para saltar directo entre nodos con `leaf` no vacío:

```
for (int u = v; u; u = next_match(u))
    for (int id : t[u].leaf) reportar(id, posicion_actual);
```



$$O\left(\sum_i |T_i|\right) \text{ construcción} \quad + \quad O(|S|) \text{ recorrido} \quad + \quad O(\text{ocurrencias}) \text{ reporte}$$

- El trie se construye **una sola vez**: $O(\sum |T_i| \cdot |\Sigma|)$; `link`, `go` y `exit` se calculan y guardan la primera vez que se piden.
- El texto se recorre en $O(|S|)$ **amortizado** con `go`, sin retroceder, igual que KMP, pero para todos los patrones a la vez.
- `next_match` comprime el fail-chain: cada match real se reporta **una vez**, sin pasar por nodos vacíos.



Código completo

```
struct Vertex {
    int next[26], go[26];
    int p, link = -1;
    int exit = -1, cnt = -1;
    vector<int> leaf;
    char pch;
    Vertex(int p=-1, char ch='$')
        : p(p), pch(ch) {
        for (int i = 0; i < 26; i++)
            next[i] = -1, go[i] = -1;
    }
};
vector<Vertex> t(1);

void add(string &s, int id) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].leaf.push_back(id);
}
```

```
int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(
                get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0
                : go(get_link(v), ch);
    }
    return t[v].go[c];
}
```

```
int next_match(int v) { // Optional
    if (t[v].exit == -1) {
        if (t[get_link(v)].leaf.size())
            t[v].exit = get_link(v);
        else
            t[v].exit = v == 0 ? 0
                : next_match(get_link(v));
    }
    return t[v].exit;
}

int cnt_matches(int v) { // Optional
    if (t[v].cnt == -1)
        t[v].cnt = v == 0 ? 0
            : t[v].leaf.size()
                + cnt_matches(get_link(v));
    return t[v].cnt;
}
```



Implementación completa

La implementación completa de Aho-Corasick usada en estas slides está disponible en:

[**github.com/kovaxis/apuntes-bella-y-sensual**](https://github.com/kovaxis/apuntes-bella-y-sensual)



Avanzado: DP sobre el autómatata

Cada nodo del autómatata es un **estado**. Se puede hacer DP sobre esos estados para contar strings enteros, no solo posiciones de un texto dado:

$dp[i][v] = \#\{\text{strings de largo } i \text{ que terminan en el estado } v, \text{ sin tocar ningún nodo "end"}\}$

$$dp[i + 1][\text{goto}(v, c)] += dp[i][v] \quad \forall c \in \Sigma$$

n chico

DP directo: $O(n \cdot \sum |T_i| \cdot |\Sigma|)$

marcar como "muertos" los estados end

n gigante (10⁹)

matriz de transición + exponenciación

$O((\sum |T_i|)^3 \log n)$

Es la técnica detrás de problemas clásicos de "contar strings que evitan un conjunto de patrones prohibidos".



F. x-prime Substrings

C. Genetic engineering

¡Fin!